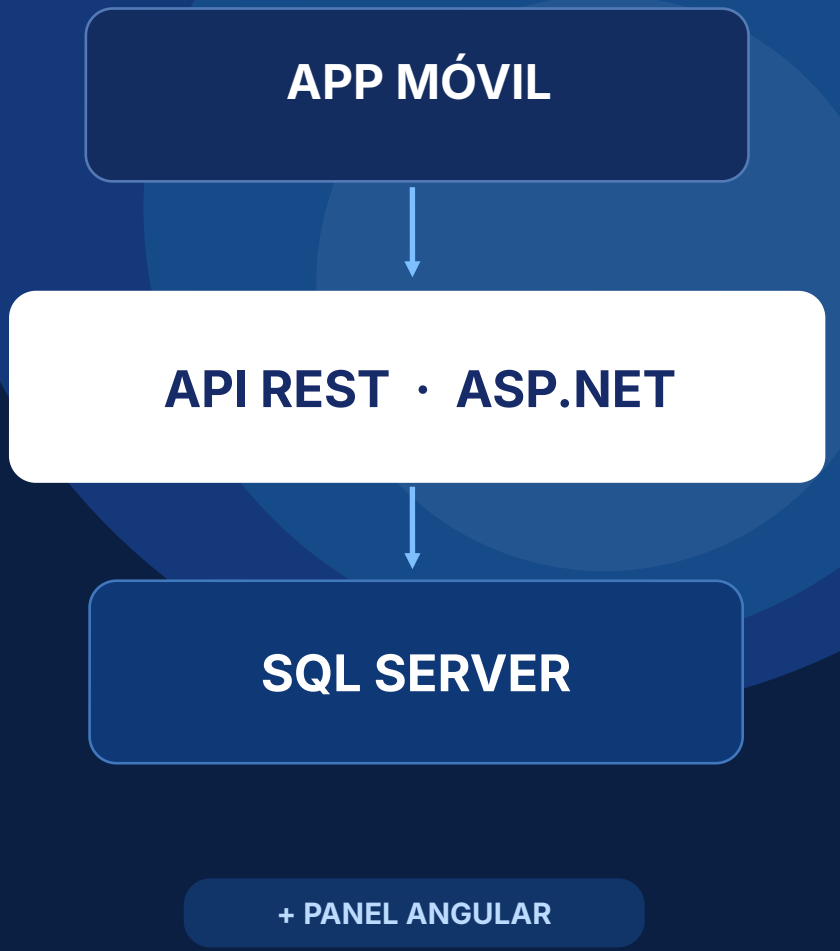
Consware

PRUEBA TÉCNICA FULLSTACK

Arquitectura de solución

Sistema de reembolso de gastos

ASP.NET Core · SQL Server · Flutter · Angular



Diseño propuesto · MVP de 4 horas · Octubre de 2026

Consware

01 / El caso de negocio

El objetivo no es construir un ERP: es resolver reembolsos con reglas claras.

01

Empleado

Registra un gasto con monto, categoría y descripción.

02

Aprobador

Revisa solicitudes pendientes y decide: aprobar o rechazar.

REGLA CENTRAL

$500.000 COP

≤ 500.000	Aprobación automática
> 500.000	Queda pendiente

RECHAZO CON MOTIVO OBLIGATORIO

ESTADOS DEFINITIVOS NO CAMBIAN

Consware

02 / Alcance de la prueba

Dos entregables principales y un panel adicional, dentro de cuatro horas.

01

API ASP.NET Core

PRIORIDAD CRÍTICA

- Crear, consultar, listar
- Aprobar y rechazar
- Validar reglas en backend
- Persistir en SQL Server

02

Flutter · Empleado

PRIORIDAD CRÍTICA

- Formulario del gasto
- Mis solicitudes por empleado
- Estado y motivo de rechazo
- Mensajes ante fallo de API

03

Angular · Aprobador

ÚLTIMO EN PRIORIDAD

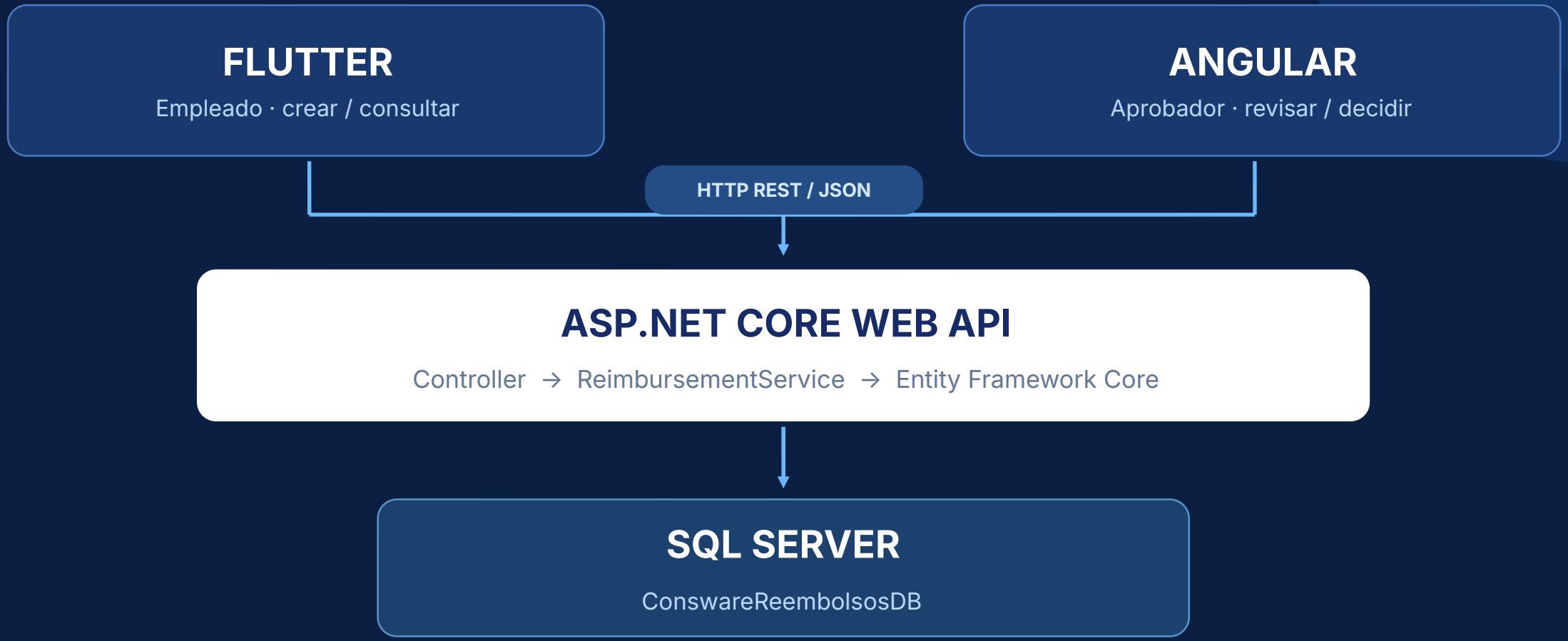
- Tabla por estado
- Pendientes por defecto
- Acciones aprobar/rechazar
- Motivo obligatorio al rechazar

FUERA DEL MVP: login · carga de comprobantes · microservicios · despliegue en nube

Consware

03 / Arquitectura extremo a extremo

Una API central, dos interfaces independientes y una sola base de datos relacional.



Los clientes nunca se conectan directamente a la base de datos.

04 / Responsabilidades por capa

La lógica de negocio vive en el backend, nunca en las interfaces.

PRESENTACIÓN

Flutter

Registra y consulta gastos

Angular

Filtra y toma decisiones

NEGOCIO

REST Controller

Rutas y respuestas HTTP

Service

Validación y política de montos

PERSISTENCIA

Entity Framework

DbContext y migraciones

SQL Server

Datos durables y relacionales

Separación simple y defendible: Controller → Service → DbContext. Sin capas innecesarias.

Consware

05 / Flujo completo de una solicitud

Desde el formulario Flutter hasta la respuesta y la decisión del aprobador.

01 — Empleado registra

Monto + categoría + descripción

02 — Flutter envía POST

JSON a /api/reimbursements

03 — Backend valida

Monto positivo y campos presentes

04 — Servicio decide

≤ $500.000 → aprobado automáticamente

05 — EF Core guarda

Registro persistido en SQL Server

06 — Consulta y revisión

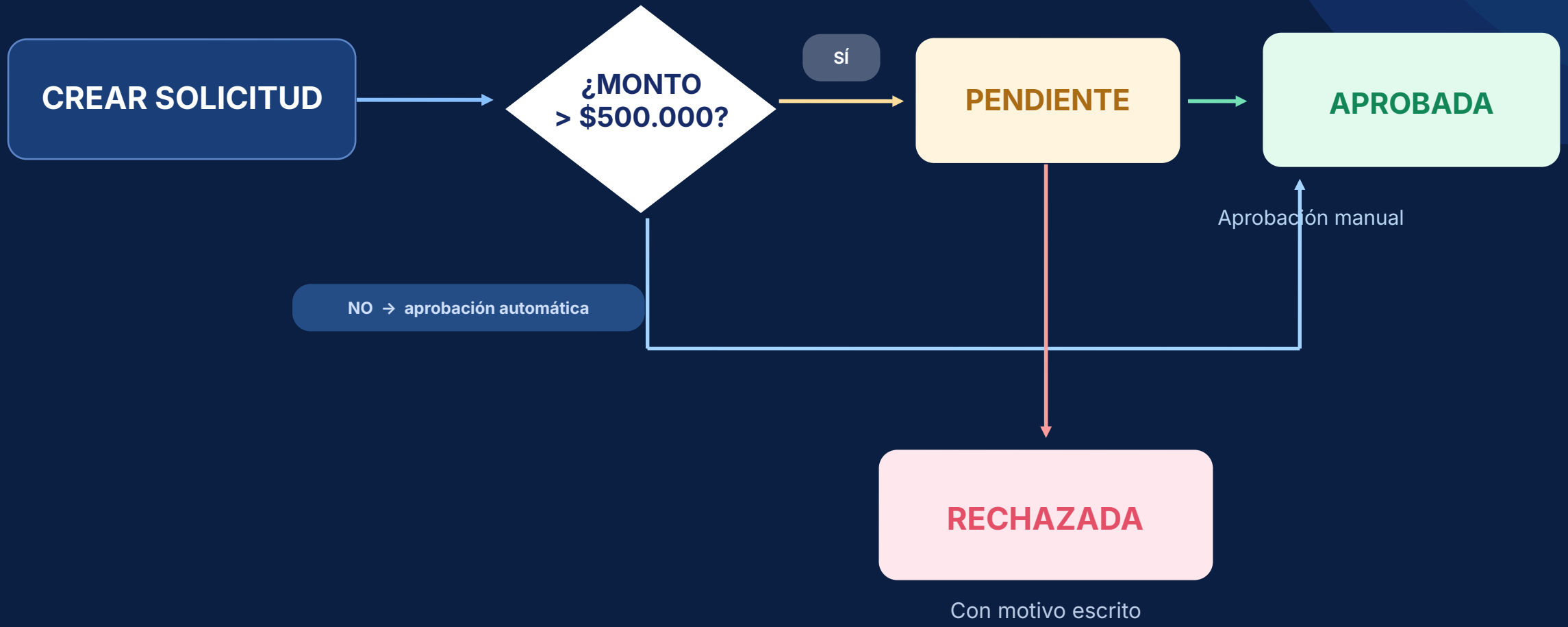
Flutter muestra estado; Angular decide pendientes

SI HAY RECHAZO: motivo obligatorio

SI YA ES FINAL: no puede cambiar

06 / Regla de estados

Tres estados posibles. Dos son definitivos.



ESTADOS DEFINITIVOS: APROBADA Y RECHAZADA NO REGRESAN A PENDIENTE

07 / Modelo de base de datos

Una tabla SQL Server. Lo necesario para que el MVP funcione y sea fácil de explicar.

dbo.ReimbursementRequests

Id	INT IDENTITY · PK
EmployeeName	NVARCHAR(100)
Amount	DECIMAL(18,2)
Category	NVARCHAR(60)
Description	NVARCHAR(500)
Status	VARCHAR(16)
RejectionReason	NVARCHAR(500) · NULL
CreatedAtUtc	DATETIME2
DecidedAtUtc	DATETIME2 · NULL

DECISIONES

01 **Una sola tabla**

02 **Empleado como texto**

03 **Categoría como texto**

04 **EF Core Migrations**

05 **Sin tabla de login**

Instancia de desarrollo: JEREMY\MSSQLSERVER2026

08 / API REST y respuestas HTTP

Los contratos son simples, verificables y compartidos entre Flutter y Angular.

POST	/api/reimbursements	Crear solicitud
GET	/api/reimbursements	Listar · filtros por estado y empleado
GET	/api/reimbursements/{id}	Detalle de una solicitud
PATCH	/api/reimbursements/{id}/approve	Aprobar pendiente
PATCH	/api/reimbursements/{id}/reject	Rechazar pendiente con motivo

CÓDIGOS QUE IMPORTAN

201	Creada
200	Operación correcta
400	Validación fallida
404	No existe
409	Estado final

Backend valida y controla

09 / Organización del repositorio

Un solo Git, tres proyectos autónomos, responsabilidades claras.

backend/

- Controllers/
- Services/
- DTOs/
- Models/
- Data/
- Migrations/
- Program.cs

mobile/

- lib/config/
- lib/models/
- lib/services/
- lib/screens/
- lib/widgets/
- main.dart
- pubspec.yaml

web/

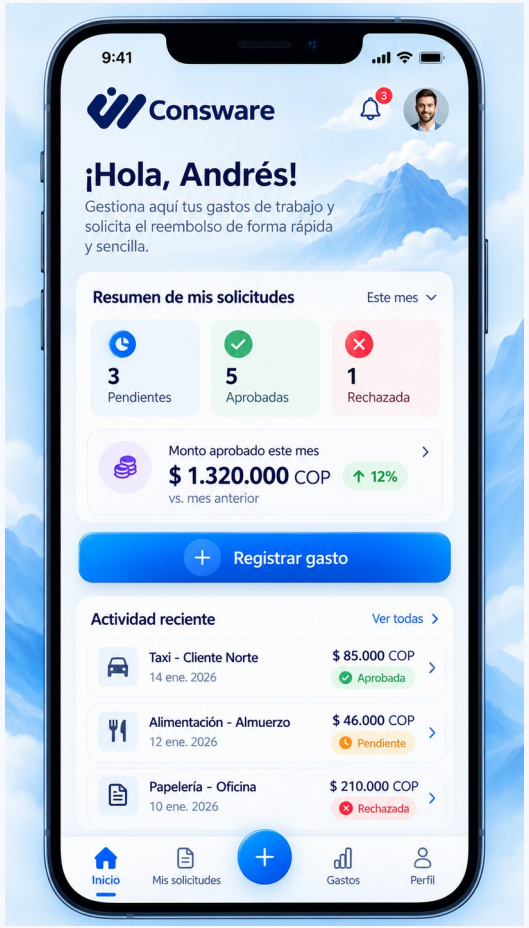
- src/app/models/
- src/app/services/
- src/app/features/
- approvals.component.ts
- approvals.component.html
- app.config.ts
- app.routes.ts

README.md + .gitignore + commits incrementales

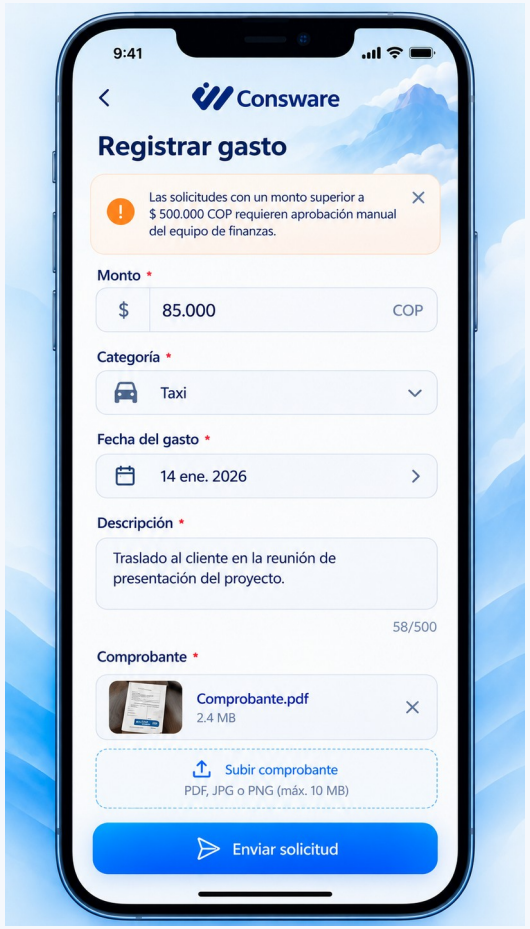
10 / Experiencia móvil · Flutter

Diseño visual de referencia para las tres vistas del empleado.

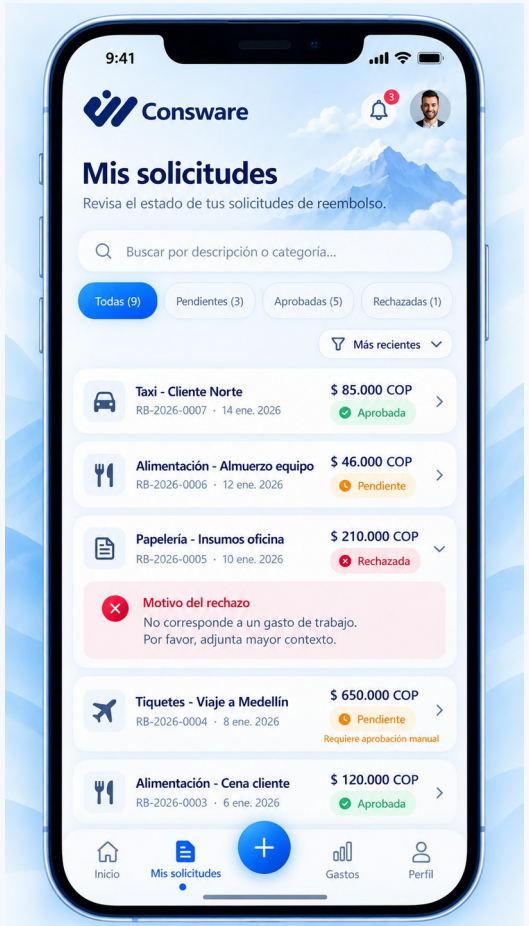
INICIO



REGISTRAR GASTO



MIS SOLICITUDES



Diseño conceptual: el MVP exige monto, categoría, descripción y consulta del estado/motivo. El backend conserva la regla de $500.000.

11 / Experiencia web · Angular

Tablero simple del aprobador: pendientes por defecto y decisiones con motivo.

Consware

Aprobaciones

Solicitudes

Configuración

Solicitudes pendientes

Pendientes	Aprobadas	Rechazadas
2	1	1

ID	Categoría	Monto	Estado
RB-001	Taxi	$650.000	Pendiente
RB-002	Tiquetes	$1.250.000	Pendiente
RB-003	Papelería	$120.000	Aprobada
RB-004	Alimentación	$780.000	Rechazada

ACCIONES

Solicitud RB-001

Taxi · $650.000 COP

✓ Aprobar

× Rechazar

Si rechaza:

Motivo obligatorio

Los estados finales bloquean las acciones.

12 / Calidad y casos críticos

Primero los límites de la regla; luego los contratos y la resiliencia.

$500.000

Aprobada

Límite incluido

$500.001

Pendiente

Revisión manual

Sin motivo

400

Rechazo inválido

ID inexistente

404

Consulta sin resultado

Estado final

409

No admite cambios

FALLA DE RED EN FLUTTER

Mostrar mensaje claro + acción de reintento. Nunca una pantalla rota.

13 / Plan de trabajo · 4 horas

La prioridad es terminar y probar; el historial Git debe mostrar avances reales.



01	**API y SQL Server**	90 min	La base debe funcionar primero.
02	**App Flutter**	90 min	Consumir una API real, no datos simulados.
03	**Panel Angular**	40 min	Solo después de tener API + móvil sólidos.
04	**README y revisión**	20 min	Comandos, decisiones, límites y uso de IA.

La guía indica ~90 min API, ~90 min Flutter y ~60 min Angular; aquí reservamos 20 min para entregar.

Consware

CIERRE / DEFENSA TÉCNICA

Una solución pequeña.
Una arquitectura clara.
Reglas que funcionan.

MVP

Cumple lo solicitado

DISEÑO

Separación de responsabilidades

EVOLUCIÓN

Login y adjuntos después

Basado en el enunciado de Consware. SQL Server y estructura interna: decisiones propuestas para la prueba.